

BD71, Big Data Spring 26

Pr. Mohamed Kas

Lab 1: Introduction to Hadoop and MapReduce

March 26, 2026

Abstract

This lab is inspired by the training "Intro to Hadoop and MapReduce" provided by Cloudera and published on Udacity. It introduces the core concepts of the Hadoop Ecosystem and MapReduce programming. You will execute basic HDFS commands and write Python scripts to process large datasets.

Contents

1	Dataset Preparation	2
2	Hadoop Basic Commands	2
3	MapReduce Job Example	2
3.1	Mapper Code	2
3.2	Reducer Code	2
3.3	Run MapReduce Job	3
3.4	Viewing the Output	3
4	Exercises: Retail Data (purchases.txt)	5
4.1	Exercise 1: Sales by Category	5
4.2	Exercise 2: Highest Individual Sale	5
4.3	Exercise 3: Payment Method Analytics	5
5	Exercises: Log Analysis (access_log)	5
5.1	Exercise 4: Page Popularity	5
5.2	Exercise 5: IP Tracking	5
5.3	Exercise 6: HTTP Status Codes	5
5.4	Exercise 7: Total Bytes Transferred	5

1 Dataset Preparation

For this lab, we will be using two large datasets. Due to the older OS on the Cloudera Quickstart VM, we must use the `-no-check-certificate` flag to bypass modern SSL certificate errors when downloading. You can download and extract them directly using the terminal:

```
1 # Download the retail purchases dataset
2 wget --no-check-certificate http://content.udacity-data.com/courses/ud617/purchases.
  txt.gz
3 gunzip purchases.txt.gz
4
5 # Download the web server access log dataset
6 wget --no-check-certificate http://content.udacity-data.com/courses/ud617/access_log.
  gz
7 gunzip access_log.gz
```

2 Hadoop Basic Commands

All commands that interact with the Hadoop system begin with `hadoop fs`. The options added are largely inspired by standard Unix commands.

- To create a directory in HDFS called `myinput`: `hadoop fs -mkdir myinput`
- To copy the file `purchases.txt` into HDFS under the directory `myinput`: `hadoop fs -put purchases.txt myinput/`
- To display the contents of the directory `myinput`: `hadoop fs -ls myinput`
- To view the last lines of the file: `hadoop fs -tail myinput/purchases.txt`

3 MapReduce Job Example

For our lab session, we are using Python to develop the Mappers and Reducers. Intermediate processing tasks (such as sorting, for example) are automatically handled by Hadoop.

3.1 Mapper Code

Our goal is to determine the total sales per store, using a log file with the following fields: Date, Time, Store, Product, Cost, Payment. To calculate the sales by store, the key-value pair to extract is (store, cost).

```
1 #!/usr/bin/env python
2 import sys
3
4 for line in sys.stdin:
5     data = line.strip().split("\t")
6     if len(data) == 6:
7         date, time, store, item, cost, payment = data
8         print "{0}\t{1}".format(store, cost)
```

Listing 1: mapper.py

3.2 Reducer Code

The Reducer is responsible for processing the desired operation on key/value pairs, which are sorted by Hadoop. Once the Mapper extracts the pairs (store, cost), the Reducer's task is to sum up all the costs for the same store.

```
1 #!/usr/bin/env python
2 import sys
3
4 salesTotal = 0
5 oldKey = None
6
7 for line in sys.stdin:
8     data_mapped = line.strip().split("\t")
9     if len(data_mapped) != 2:
10         continue
11
12     thisKey, thisSale = data_mapped
13
14     if oldKey and oldKey != thisKey:
15         print "{0}\t{1}".format(oldKey, salesTotal)
16         salesTotal = 0
17
18     oldKey = thisKey
19     salesTotal += float(thisSale)
20
21 if oldKey != None:
22     print "{0}\t{1}".format(oldKey, salesTotal)
```

Listing 2: reducer.py

3.3 Run MapReduce Job

To launch an entire job on Hadoop, it involves calling the mapper and then the reducer on a large input, and obtaining the result directly on HDFS. Notice the use of the `-files` argument to distribute our Python scripts to the cluster nodes.

```
1 hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \
2 -files mapper.py,reducer.py \
3 -mapper mapper.py -reducer reducer.py \
4 -input myinput -output joboutput
```

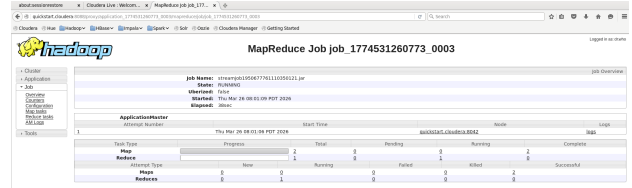
3.4 Viewing the Output

After the job completes successfully, the output directory will contain a file named `part-00000`, representing the desired result. Note that the output directory must not exist before executing the command. To view the final output data directly in your terminal, use the `-cat` command:

```
1 hadoop fs -cat joboutput/part-00000
```

```
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ hdfs fs / jar://lib/hadoop-mapreduce-hadoop-streaming-2.6.0-0.0.jar /tmp/reducer.py --reducer reducer.py --input input.txt --output output.txt
26/03/26 08:01:04 INFO mapreduce.Job: map 26% reduce 0%
26/03/26 08:01:04 INFO mapreduce.Job: map 50% reduce 0%
26/03/26 08:01:04 INFO mapreduce.Job: map 57% reduce 0%
26/03/26 08:01:04 INFO mapreduce.Job: map 67% reduce 0%
26/03/26 08:01:04 INFO mapreduce.Job: map 83% reduce 0%
26/03/26 08:01:04 INFO mapreduce.Job: map 100% reduce 0%
26/03/26 08:01:04 INFO mapreduce.Job: map 100% reduce 100%
26/03/26 08:01:04 INFO mapreduce.Job: Job job_1774531260773_0003 completed successfully
26/03/26 08:01:04 INFO mapreduce.Job: Counters: 49
File System Counters
FILE: Number of bytes read=75658848
FILE: Number of bytes written=151703186
FILE: Number of read operations=0
FILE: Number of large read operations=0
FILE: Number of write operations=0
HDFS: Number of bytes read=211317258
HDFS: Number of bytes written=2193
HDFS: Number of read operations=9
HDFS: Number of large read operations=0
HDFS: Number of write operations=2
Job Counters
Launched map tasks=2
Launched reduce tasks=1
Data-local map tasks=2
Total time spent by all maps in occupied slots (ms)=59384
Total time spent by all reduces in occupied slots (ms)=12096
Total time spent by all map tasks (ms)=59384
Total time spent by all reduce tasks (ms)=12096
Total vcore-milliseconds taken by all map tasks=59384
Total vcore-milliseconds taken by all reduce tasks=12096
Total megabyte-milliseconds taken by all map tasks=60809216
Total megabyte-milliseconds taken by all reduce tasks=12386304
Map-Reduce Framework
Map input records=4138476
Map output records=4138476
Map output bytes=67381890
Map output materialized bytes=75658854
Input split bytes=238
Combine input records=0
Combine output records=0
Reduce input groups=103
Reduce shuffle bytes=75658854
Reduce input records=4138476
Reduce output records=103
Spilled Records=8276952
Shuffled Maps =2
```

(a) Execution Step 1



(b) Execution Monitoring

```
[cloudera@quickstart lab 1]$ hadoop fs -cat joboutput_example/part-00000
Albuquerque 10052311.42
Anaheim 10076416.36
Anchorage 9933500.4
Arlington 10072207.97
Atlanta 9997146.7
Aurora 9992970.92
Austin 10057158.9
Bakersfield 10031208.92
Baltimore 10096521.45
Baton Rouge 10131273.23
Birmingham 10076606.52
Boise 10039166.74
Boston 10039473.28
Buffalo 10001941.19
Chandler 9919559.86
Charlotte 10112531.34
Chesapeake 10038504.92
Chicago 10062522.07
Chula Vista 9974951.34
Cincinnati 10139505.74
Cleveland 10067835.84
Colorado Springs 10061105.87
Columbus 10035241.03
Corpus Christi 9976522.77
Dallas 10066548.45
Denver 10031534.87
Detroit 9979260.76
Durham 10153890.21
El Paso 10016409.97
Fort Wayne 10132594.02
Fort Worth 10120830.65
Fremont 10053242.36
Fresno 9976260.26
Garland 10071043.92
Gilbert 10062115.19
Glendale 10044493.97
Greensboro 10033781.39
Henderson 10053416.05
Hialeah 10047052.76
Honolulu 10006273.49
Houston 10042106.27
Indianapolis 10090272.77
Irvine 10084867.45
Irving 10133944.08
```

(c) Hadoop job progress

(d) Final Output Data

Figure 1: MapReduce Job Execution and Output Visualization.

4 Exercises: Retail Data (`purchases.txt`)

The `purchases.txt` dataset contains millions of retail transactions. The goal is to write your own Mappers and Reducers to extract valuable business intelligence.

4.1 Exercise 1: Sales by Category

Extract the list of total sales by product category.

- What is the total sales value for the **Toys** category?
- What is the total sales value for the **Consumer Electronics** category?

4.2 Exercise 2: Highest Individual Sale

Extract the highest single sales amount recorded for each store.

- What is the maximum sale value for **Reno**, **Toledo**, and **Chandler**?

4.3 Exercise 3: Payment Method Analytics

Determine the total revenue generated through each payment method (e.g., Visa, MasterCard, Cash).

- Which payment method generated the most revenue overall?

5 Exercises: Log Analysis (`access_log`)

The `access_log` file stores accesses to various pages of a website. An entry in this file consists of several fields separated by spaces:

```
10.223.157.186 - - [15/Jul/2009:15:50:35 -0700] "GET /assets/js/lowpro.js HTTP/1.1" 200 10469
```

5.1 Exercise 4: Page Popularity

Write a MapReduce job to provide the number of times each page of the website has been accessed.

- How many times has the page `/assets/js/theassociates.js` been accessed?

5.2 Exercise 5: IP Tracking

Write a MapReduce job to calculate the total number of site accesses per IP address.

- How many times has the IP address `10.99.99.186` accessed the site?

5.3 Exercise 6: HTTP Status Codes

Web administrators need to know if the site is generating errors. Write a MapReduce job to count the occurrences of each HTTP status code (e.g., 200, 404, 304).

- How many times did a "404 Not Found" error occur in the log?

5.4 Exercise 7: Total Bytes Transferred

Calculate the total number of bytes transferred back to each IP address. The bytes transferred is the final number in each log row (e.g., 10469 in the example above). *Hint: Some requests have a "-" instead of a byte size. Your code must handle this gracefully.*